Assignment 5: Optimising Attention on a CPU

CS6886: Systems for Deep Learning, IIT Madras. Anurag Dhiman. Measured on an Apple M4 MacBook Air, November 2026.

On this machine the given attention code goes from 1,084 ms to **6.56 ms on one thread (165× faster)** and to **2.61 ms with OpenMP (416× faster)** at N = 4096, d = 64. All variants pass `make check`, with no BLAS or math library linked. Table 1 gives the results; the rest of this report explains each one with measurements.

Variant	What changed	Time (ms)	GFLOP/s	Speedup	AI (FLOP/B)
naive	As given	1,084	3.96	1	12.6
loops	Loop reordering, K^T copy	168	25.5	6.4×	12.5
tiled	Register and cache tiling, 16 accumulators	67.4	63.7	16.1×	12.5
fused	One row of S at a time	116	36.9	9.3×	814
best	SME2 FMOPA, NEON exp, 16-row fusion	6.56	655	165×	546
omp, 10 threads	attn_best over row blocks	2.61	1,648	416×	273

Table 1. Every variant at N = 4096, d = 64. Speedup is over naive; AI is arithmetic intensity (Q5).

Three findings shaped the design, and each came from a measurement rather than an assumption:

- **The bottleneck in naive is FP addition order, not cache misses.** Clang 23 already vectorises the multiplications, but without `-ffast-math` the additions stay in a serial chain. Step 3 runs at the full 3.05-cycle FMADD latency per element.
- **Matrix products are sums of outer products, which is what SME2 FMOPA computes.** It reaches 2,014 GFLOP/s, 14× the NEON peak, but vector arithmetic in streaming mode is 4.5× slower than NEON, so the best code switches between the two.
- **Cache tiling and DRAM traffic hardly matter here.** A 16 MiB L2 and 70 GB/s from DRAM remove the constraints those techniques target. Register tiling gives the gain, and storing the N × N matrix costs only 5%.

How I measured

- All timings come from the `make` targets with the driver unchanged (`diff` confirms it is byte-identical to the starter in all six files). Compiler: Homebrew clang 23.1.4 with `-O3 -march=native`, as the Makefile asks; `CC` is the only change to the Makefile.
- Each reported time is the median over 5 rounds of the driver's own median of 5, with pauses between runs. The machine ran on battery with Low Power Mode off, and macOS recorded no thermal warnings. Rounds usually agree within 5%. Isolated outliers do occur: outside these rounds, about 40 further runs of best at N = 4096, d = 64 gave 6.55 to 7.15 ms, and one gave 13.2 ms. I did not establish the cause, though the SME unit is shared and another process using it would explain it.
- GFLOP/s uses the driver's count, $4N^2d$: $2N^2d$ for each matrix product. **Softmax is not counted**, at roughly 15 FLOPs per element, or about 6% more at d = 64. Softmax time therefore lowers the reported GFLOP/s without adding to it.
- **Clock warm-up (disclosure).** A cold process needs about 100 ms to ramp up: sampling a chain of 1-cycle additions from process start gives 1.6 GHz at 19 ms, 2.6 GHz at 40 ms and 3.5 GHz at 64 ms, stepping through the DVFS table reported by `ioreg`. The driver warms up with one call, so a fast variant is timed during the ramp, while naive is not. This biases the comparison against fast variants and breaks comparison with a peak computed at 4.41 GHz. Since the driver must stay unchanged, each non-naive `attention()` repeats its untimed first call until 200 ms have passed. No timed call changes. `-DNO_CLOCK_WARMUP` removes this, and Table 2 gives both sets of numbers. It matters mainly at N = 1024, the first input `make` runs (0.91 → 0.53 ms for best). At N = 4096 it changes the result by under 3%, so the headline numbers do not depend on it.

Variant	N=1024, d=64	no warm-up	N=2048, d=64	no warm-up	N=4096, d=64	no warm-up	N=4096, d=128	no warm-up
loops	10.5	11.2	41.0	41.0	168	168	297	297
tiled	4.21	5.42	16.1	16.0	67.4	68.1	105	106
fused	7.38	8.57	29.1	29.0	116	115	221	218
best	0.53	0.91	1.83	2.52	6.56	6.70	9.40	9.70
omp, 10 threads	0.31	0.37	0.83	0.94	2.61	2.55	5.08	4.96

Table 2. Time in ms with the first-call warm-up (used throughout this report) and without it. Later inputs benefit less because make runs them straight after earlier ones, once the clock has already risen.

Q1. My machine

`machine_spec.txt` has every field. Where macOS or Apple does not state a value, I measured it with microbenchmarks (Appendix A); Table 3 shows where each number came from.

Field	Value	Source
CPU	Apple M4 (MacBook Air, Mac16,12), family "Donan"	sysctl, SDK machine.h
Cores	10 physical, 10 logical (no SMT): 4 performance + 6 efficiency	sysctl hw.perflevel*
Clocks	No published base clock. DVFS tops: P 4.464 GHz, E 2.892 GHz	ioreg voltage-states
Clock, 1 or 2 P-cores busy	4.41 GHz	measured, ALU chain
Clock, all 10 cores busy	P 3.82 GHz, E 2.34 GHz	measured, ALU chain
SIMD	NEON 128-bit (4 FP32 lanes). SME2: streaming SVE 512-bit (16 lanes), ZA = 4 tiles of 16 × 16	sysctl hw.optional, rdsvl
FMA units	P-core 4, E-core 2 (128-bit); one SME unit per cluster	measured, Table 4
L1D / L2 / L3	P: 128 KiB / 16 MiB shared by 4. E: 64 KiB / 4 MiB shared by 6. No CPU L3	sysctl
Cache line	128 bytes	sysctl hw.cachelinesize
RAM	16 GB LPDDR5X-7500, 128-bit bus (macOS labels it LPDDR5)	system_profiler, published specs
Software	macOS 15.5; Homebrew clang 23.1.4	sw_vers, clang --version

Table 3. Machine summary.

Peak FP32 rate

One FMA is 2 FLOPs per lane. On a P-core: **4.41 GHz × 4 FMA units × 4 lanes × 2 = 141.1 GFLOP/s**. I measured 141.3 GFLOP/s, so the 4 FMA units are real.

All cores, using the clocks measured with all of them busy: 4 × (3.82 × 4 × 4 × 2) + 6 × (2.34 × 2 × 4 × 2) = 489.0 + 224.6 = **713.6 GFLOP/s**. I measured 697.7 GFLOP/s, which is 98% of that, and 676 GFLOP/s after 30 s. With the top DVFS clocks the same formula gives 842, which this fanless machine cannot sustain.

SME2 needs its own ceiling. One FMOPA adds a 16 × 16 outer product, 16 × 16 × 2 = 512 FLOPs. I measured 3.93 FMOPA/ns across 4 ZA tiles, so **2,014 GFLOP/s** on one thread. The FMOPA rate does not grow with threads: the total stays at **2,350 GFLOP/s** (about 1985 from the P cluster, 363 from the E cluster), which shows that each cluster shares one unit. I give "% of peak" against the NEON peak unless stated, because that is the conventional calculation, and show the SME peak beside it for the variants that use SME.

Memory bandwidth and ridge point

7500 MT/s × 16 bytes = **120 GB/s**, which matches Apple's published figure. Measured reads reach 70 GB/s on one thread, 74 GB/s from the P cluster and 109 GB/s with all cores (91% of the theoretical figure). The P cluster's fabric link, not the DRAM, appears to cap a single cluster.

Ridge point = peak / 120 GB/s	Peak (GFLOP/s)	Ridge (FLOP/B)
NEON, one core	141.1	1.18
NEON, all cores	713.6	5.95
SME2, one thread	2,014	16.8
SME2, both clusters	2,350	19.6

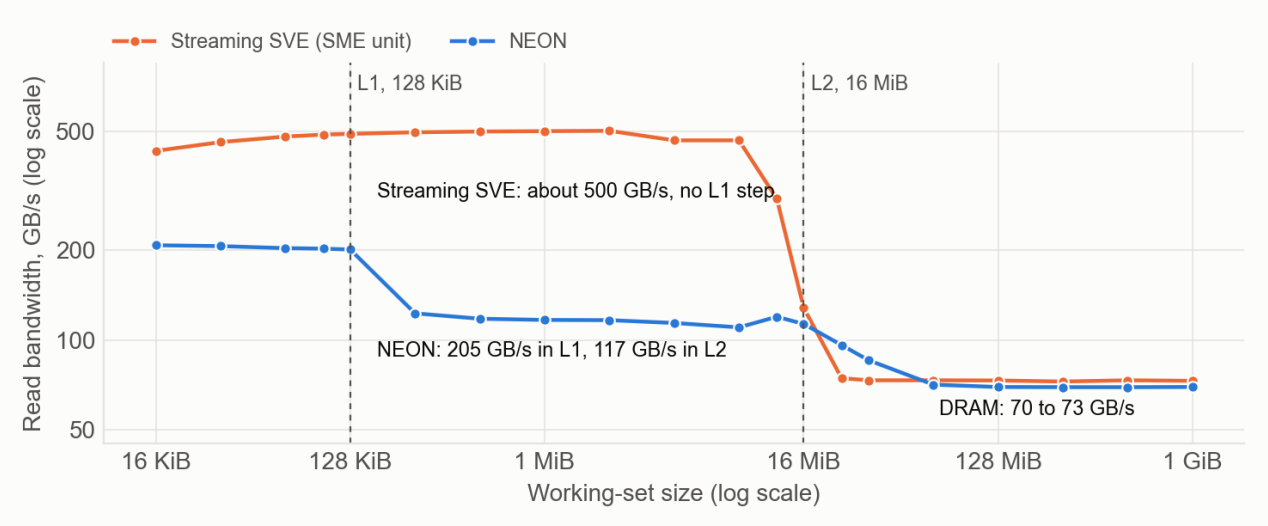


Figure 1. Read bandwidth against working-set size on a P-core. Streaming SVE reads see no L1 step at all and fall only as the data leaves L2, so the SME unit behaves as if it were attached at L2.

Measurement	Result	Why it matters
FMLA, by independent accumulators (P-core)	k=1: 0.33, k=4: 1.19, k=8: 2.06, k=12: 2.90, k=16: 4.01 per cycle	Latency × units predicts 12 accumulators. In practice it takes 16, so the kernels use 16
FMLA, by independent accumulators (E-core)	k=1: 0.33, k=4: 1.09, k=8: 2.00, k=16: 2.00 per cycle	E-core has 2 FMA units
Scalar latency	FMADD 3.05, FMUL 3.05, FADD 2.10 cycles	Sets the cost of an in-order reduction
FMOPA throughput	1 tile 0.98, 2 tiles 1.97, 4 tiles 3.93 per ns	Use all 4 tiles: 503 → 2014 GFLOP/s
Streaming SVE FMLA	0.98 per ns, 31 GFLOP/s	4.5× slower than NEON, so softmax must leave streaming mode
libm expf	0.92 ns per call	Cannot be vectorised by the compiler
smstart + smstop	13 to 17 ns	Switching mode per block is negligible

Table 4. Microbenchmarks. Each result comes from a dedicated loop (Appendix A); instructions per cycle use the clock measured around the loop.

Q2. Naive baseline

Input	Time (ms)	GFLOP/s	% of peak (141.1)
N=1024, d=64	62.3	4.31	3.05
N=2048, d=64	259	4.14	2.93
N=4096, d=64	1,084	3.96	2.81
N=4096, d=128	2,255	3.81	2.70

Time against N. Doubling N multiplies the time by 4.17 and then 4.18, close to the 4× of an N^2 algorithm. The small excess is a cache effect, shown below. **Time against d.** Doubling d at N = 4096 multiplies it by 2.08. It should be just under 2, since softmax does not depend on d, but step 1 grows by 2.78× for the reason given below.

Input	Step 1, QK^T (ms)	Softmax (ms)	Step 3, SV (ms)	Step 3 share	Step 1, cycles/element	Step 3, cycles/element
N=1024, d=64	10.7	1.75	51.3	81%	0.70	3.37
N=2048, d=64	42.0	7.04	216	82%	0.69	3.55
N=4096, d=64	171	28.3	890	82%	0.70	3.66
N=4096, d=128	477	28.2	1,780	78%	0.98	3.66

Table 5. Where naive spends its time. Cycles per element are at 4.41 GHz; an element is one multiply and one add.

Where the time goes. Step 3 takes about 80% of the time although it does the same FLOPs as step 1, and softmax takes under 3%. I expected cache misses to explain step 3: V is walked with a 256-byte stride, so every element touches a new 128-byte line. The data rejected that. Shrinking V to fit in L1 (N = 128, 32 KB) still leaves step 3 about three times slower than step 1. The generated code gives the real explanation:

- **Step 3 is a scalar FMADD chain.** The compiler does emit a vector loop, but it is guarded by a `d == 1` stride check, so at d = 64 the scalar path runs. Each element waits for the previous `acc`, and the chain length is N, far longer than the out-of-order window, so the chain alone costs 3.05 cycles per element: 2 FLOPs per 3 cycles against a peak of 32 FLOPs per cycle, or 2% of peak. Cache misses add to this only as the column of V outgrows L1: I measured 3.37 cycles at N = 1024, where one column spans 131 KB of lines, about the size of L1, and 3.66 at N = 4096, where it spans 524 KB.
- **Step 1 uses ordered reductions.** Clang vectorises the multiplies and adds the 16 products serially with FADD, which keeps the result exact without `-ffast-math`. The chain costs 2.10 cycles per add in isolation, yet step 1 takes only 0.70. Each dot product resets `acc = 0`, so consecutive j are independent, and the core runs several chains at once: I measured 40.5 cycles per d = 64 dot product against a 134-cycle chain, about 3.3 in flight.

d	16	32	64	64	128	256
N	1024	1024	1024	4096	4096	1024
Step 1, cycles/element	0.55	0.55	0.70	0.70	0.98	1.25

Table 6. The evidence for the out-of-order explanation. A longer chain means fewer chains fit in the window, so the cost per element climbs toward the 2.10-cycle chain limit. This is why doubling d more than doubles step 1.

Q3. Optimisations

Every variant passes `make check` on every input; the error column in each table is its measure, with a limit of 1e-4. Table 7 shows where each variant spends its time, and the sections below refer to it.

Variant	Other	Step 1	Softmax	Step 3	Total	Matrix-product GFLOP/s	Softmax share
naive	0.00	171	28.3	890	1,090	4.05	3%
loops	1.19	69.5	20.5	77.8	169	29.2	12%
tiled	0.00	22.9	20.5	23.9	67.3	91.9	30%
fused	0.00	53.8	19.8	42.2	116	44.8	17%
best	0.13	1.80	2.90	1.80	6.63	1,193	44%

Table 7. Time per phase in ms at N = 4096, d = 64, from instrumented copies of the variants. "Other" is packing and setup. Matrix-product GFLOP/s is $4N^2d$ over steps 1 and 3 only.

(a) attn_loops.c: loop reordering

Input	Time (ms)	GFLOP/s	% of NEON peak	Speedup over naive	make check	Error
N=1024, d=64	10.5	25.5	18.1	5.9×	PASS	1.1e-06
N=2048, d=64	41.0	26.2	18.5	6.3×	PASS	8.4e-07
N=4096, d=64	168	25.5	18.1	6.4×	PASS	1.2e-06
N=4096, d=128	297	28.9	20.5	7.6×	PASS	1.7e-06

What it does. Moving the output index innermost turns each reduction into an element-wise update, `S[i][j] += q * K[j][k]`, which has no ordering constraint and vectorises freely. Step 3 becomes i-j-k, contiguous in

both V and O, and gets 11× faster. In step 1 the inner index strides through K, so **reordering alone makes it worse**: 3.6 GFLOP/s, 4× slower than naive's dot products, because NEON has no gather and the loads are scalarised. A transposed copy of K ($N \cdot d$ moves, negligible next to $N^2 d$) makes the inner loop contiguous and gives 32 GFLOP/s. I also wrote the max as `fmaxf`, which vectorises where the `if` form does not, and that alone made the scan 9× faster.

What limits it now. Each FMA needs two loads and a store. At 3 loads per cycle (the 205 GB/s L1 read rate) that caps it at 1.5 FMA per cycle, and the accumulators live in memory rather than registers. K^T and V stream from L2 at 0.48 FLOP/B (Q5), which at the measured 117 GB/s caps the products near 56 GFLOP/s; they reach 29. Softmax is 12% of the time.

(b) attn_tiled.c: cache and register tiling

Input	Time (ms)	GFLOP/s	% of NEON peak	Speedup over naive	make check	Error
N=1024, d=64	4.21	63.8	45.2	14.8×	PASS	8.7e-07
N=2048, d=64	16.1	66.8	47.4	16.1×	PASS	1.1e-06
N=4096, d=64	67.4	63.7	45.2	16.1×	PASS	1.6e-06
N=4096, d=128	105	81.6	57.9	21.4×	PASS	1.6e-06

Register tiling. Accumulators move from memory into 16 registers, the number Table 4 showed is needed for 4 FMA per cycle. Step 1 uses a 4 × 4 kernel on Q and K rows, needing no transposed copy, with 8 loads per 16 FMAs. Pairwise additions turn the accumulators into S at the end. Step 3 uses a 4 × 16 kernel in which one S register supplies 4 multipliers through FMLA-by-element.

Cache tiling. I sized the tiles for the 128 KiB L1 (TILE_I = 64 rows, TILE_J = 256 rows of K and V), and then swept them. **Cache tiling has no measurable effect on this machine:**

TILE_I × TILE_J (rows)	64 × 64	64 × 256	64 × 512	64 × 2048	64 × 8192 (no tile)	4 × 8192	1024 × 256
Step 1 (ms)	22.4	22.5	22.9	22.8	22.8	22.8	22.5
Step 3 (ms)	23.3	24.1	23.7	23.2	23.3	23.0	24.1

Table 8. Tile-size sweep at N = 4096, d = 64.

This has a clear explanation. The 4 × 4 kernel reads K at 64 bytes per 4 cycles, about 70 GB/s at 4.41 GHz, which is under the 117 GB/s measured from L2. K and V together (2 MB) sit comfortably inside the 16 MiB L2. So register tiling alone already removes the L1 constraint that cache tiling exists to fix. I kept the tiles because the assignment asks for them and they would matter for matrices beyond L2.

What limits it now. Softmax is 30% of the time, because libm's scalar expf costs 0.92 ns per element. The products reach 92 GFLOP/s, 65% of peak. Step 1 loses its share to horizontal sums and kernel entry, about 30 cycles on each 64-cycle kernel. Step 3 sustains only about 2.6 FMLA per cycle even in its long steady loop, and **I could not fully explain that**: by-element FMLA alone reaches 3.64 per cycle, and neither the tile size nor reusing the S buffer (3% saved, which also rules out page faults) changes it much.

(c) attn_fused.c: row-by-row fusion

Input	Time (ms)	GFLOP/s	% of NEON peak	Speedup over naive	make check	Error
N=1024, d=64	7.38	36.4	25.8	8.4×	PASS	2.6e-06
N=2048, d=64	29.1	36.9	26.1	8.9×	PASS	3.5e-06
N=4096, d=64	116	36.9	26.2	9.3×	PASS	6.0e-06
N=4096, d=128	221	38.9	27.6	10.2×	PASS	4.5e-06

What it does. For each row i, the code computes one row of scores, its softmax and the output row, and then discards the scores. Only N floats (16 KB) are live, so S never reaches memory, and the DRAM intensity rises from 12.5 to 814 FLOP/B. Fusion also removes the softmax division: V is weighted by raw exponentials and O[i] is scaled once, so N divisions become d multiplications.

What limits it now. It is 1.7× *slower* than tiled at N = 4096. With one row of Q there is nothing to reuse, so K and V are swept once per row and every FMA needs a new vector from L2: 8.9 GB of traffic, 0.48 FLOP/B. At 117 GB/s that caps the products at 56 GFLOP/s, and they reach 45. The trade is a clear one: fusion frees DRAM, which was not the bottleneck on this machine, and gives up L2-to-L1 reuse, which was.

Fusing a tile of rows instead keeps S out of memory without giving up the reuse. I tested this with tiled's kernels. It also measures what storing S costs:

Rows fused at once, R	1 (attn_fused)	4	16	64	256	1024	4096 (S fully stored)
Time (ms)	116	58.9	58.4	59.1	59.2	59.2	62.1

Table 9. Fusing R rows at a time, N = 4096, d = 64 (experiment, not a submitted variant). R = 4096 stores all of S, so comparing it with R = 4 gives the cost of storing S: about 3 ms, or 5%.

`attn_fused.c` stays at R = 1 because the assignment asks for row-by-row fusion. The lesson carries into `attn_best.c`, which fuses 16 rows at a time.

(d) attn_best.c: SME2 outer products and a NEON softmax

Input	Time (ms)	GFLOP/s	% of NEON peak	% of SME peak	Speedup over naive	make check	Error
N=1024, d=64	0.53	503	356.2	25.0	117×	PASS	3.4e-06
N=2048, d=64	1.83	586	415.5	29.1	142×	PASS	4.1e-06
N=4096, d=64	6.56	655	464.1	32.5	165×	PASS	5.9e-06
N=4096, d=128	9.40	914	647.7	45.4	240×	PASS	5.7e-06

The key idea. Matrix products are sums of outer products: $QK^T = \Sigma_k\, Q[:,k]\, K[:,k]^T$ and $SV = \Sigma_j\, S[:,j]\, V[j,:]$. FMOPA adds one 16 × 16 outer product to a ZA tile in one instruction, 512 FLOPs. The design follows from the measurements:

- **Fusion over 16-row blocks.** Each block goes score block → softmax → output block, with a 16 × N buffer (256 KB). S is never stored, and K and V are reused by all 16 rows.
- **Packing.** FMOPA needs Q[:,k] and K[:,k], which are columns. K is packed once per call and each Q block as it is used. The ZA array does the transposition: load rows horizontally, read columns vertically. V needs no packing, since row j is already the operand that step 3 needs.
- **Four tiles in parallel.** One tile manages 503 GFLOP/s and four manage 2014, so each pass drives all four. K comes in groups of 64 rows, and O in 64 columns.
- **Softmax on NEON.** Streaming FMLA is 4.5× slower than NEON, and a mode switch costs only about 15 ns, so softmax leaves streaming mode. Lanes hold different rows of the same column, so max, exp and sum are all element-wise.
- **A polynomial exp, 7.5× faster than libm.** Adding 1.5×2^{23} to x / ln 2 leaves the rounded integer in the low mantissa bits, giving both n and the exponent bits without FRINT or FCVT. A fused $x - n \cdot \ln 2$ needs only one constant: its error is about $2 \times 10^{-9}|n|$.
- **No max scan when it is provably unnecessary.** $\exp(s - m) / \Sigma \exp(s - m)$ does not depend on m, so the scan only prevents overflow. It costs a full pass over data that must return from L2. The bound $|s| \le d \cdot \max|Q| \cdot \max|K| \cdot$ scale proves when it can go, and a guarded path with the scan and an underflow mask covers everything else.
- **STR ZA read-out.** It is 2× faster than ST1W (0.244 vs 0.488 ns per slice). It stores horizontal slices, so K is the first FMOPA operand, which makes each slice exactly the column that step 3 needs.

```
for (int k = 0; k < d; k++) {                          /* one block of scores, 64 columns */
    const svfloat32_t q = svld1_f32(all, QP + k * vl);
    const float *x = kp + k * G;
    svmopa_za32_f32_m(0, all, all, svld1_f32(all, x),          q);
    svmopa_za32_f32_m(1, all, all, svld1_f32(all, x + vl),     q);
    svmopa_za32_f32_m(2, all, all, svld1_f32(all, x + 2 * vl), q);
    svmopa_za32_f32_m(3, all, all, svld1_f32(all, x + 3 * vl), q);
}
```

Version of attn_best (development order)	Total (ms)	Softmax (ms)
FMOPA, ST1W read-out, FRINT-based degree-5 exp, max scan	10.06	6.34
+ one-constant exp, STR ZA, no max scan when safe	7.37	3.74
+ degree-4 polynomial (exp error 2.8e-6 vs 1.7e-7; make check error unchanged at 6e-6)	6.56	2.90

Table 10. What each refinement gained at N = 4096, d = 64. Software prefetch gave nothing, so it is not used.

Why it works. The speedup over tiled comes from FMOPA (products at 1,193 vs 92 GFLOP/s) and the exp (2.9 vs 20.5 ms). Overall it reaches 655 GFLOP/s, 4.6× the NEON peak.

What limits it now. The FMOPA phases are near their real limit. FMOPA and loads share one resource: alone, FMOPA takes 0.254 ns and a load 0.128 ns, so a pair should take 0.382 ns, and I measured 0.40. With at least one load per FMOPA that gives 1340 GFLOP/s, and the phases reach 1,193. Softmax is the larger limit at 44% of the time. It costs 0.17 ns per element, about 1.5× the 0.12 ns of the same arithmetic on data already in L1. That gap is consistent with SME writes going to L2, since NEON reads such data at 1.18 ns per 64 bytes against 0.73 ns otherwise, and each line must then be written back. Software prefetching does not help.

Q4. OpenMP

`attn_omp.c` uses `attn_best.c`'s kernels unchanged. Row blocks are independent, so the parallel loop runs over them, each thread with its own score buffer, and K is packed once beforehand. Speedup is relative to `attn_omp` on one thread; efficiency is speedup / threads.

Threads	N=1024, d=64: ms	S	E	N=2048, d=64: ms	S	E	N=4096, d=64: ms	S	E	N=4096, d=128: ms	S	E
1	0.53	1.00	1.00	1.83	1.00	1.00	6.55	1.00	1.00	9.40	1.00	1.00
2	0.37	1.43	0.71	1.22	1.50	0.75	5.39	1.22	0.61	7.82	1.20	0.60
3	0.33	1.60	0.53	1.10	1.66	0.55	3.84	1.70	0.57	6.27	1.50	0.50
4	0.29	1.79	0.45	1.02	1.79	0.45	3.53	1.85	0.46	6.03	1.56	0.39
5	0.29	1.83	0.37	0.91	2.02	0.40	3.04	2.16	0.43	5.05	1.86	0.37
6	0.28	1.89	0.31	0.86	2.12	0.35	2.92	2.24	0.37	5.17	1.82	0.30
8	0.28	1.87	0.23	0.84	2.19	0.27	2.63	2.49	0.31	5.26	1.79	0.22
10	0.31	1.72	0.17	0.83	2.20	0.22	2.61	2.51	0.25	5.08	1.85	0.19

Table 11. Time, speedup (S) and parallel efficiency (E). make check passes at every thread count and input.

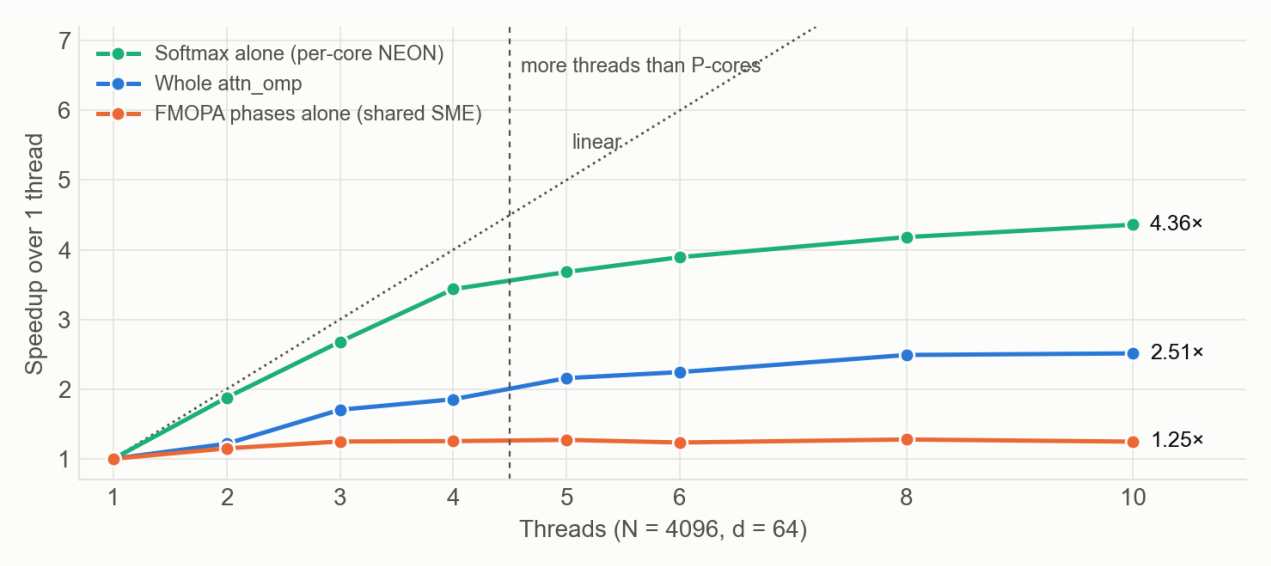


Figure 2. The two kinds of work scale differently. Each curve is relative to its own single-thread time. "FMOPA phases alone" skips the softmax; "softmax alone" runs exp and sum only.

Explaining the scaling. The speedup is poor for a reason I can measure directly: the program has two kinds of work with very different scaling.

- **FMOPA phases saturate near 1.25×.** Each cluster has one SME unit, shared by its cores, so more threads on a cluster do not add FMOPA throughput. The microbenchmark shows this plainly: 2 threads get 2009 + 363 GFLOP/s, 4 threads 990 + 990 + 182 + 182, and the total stays at about 2350 from 2 to 10 threads. The gain above one thread comes from the E-cluster's smaller unit. The per-thread rates show macOS placing the second SME thread there on its own, which explains the low speedup at t = 2.
- **Softmax scales to 4.36×.** It runs on per-core NEON units. It reaches 3.43× on 4 threads, close to 4 P-cores at the lower 4-core clock (4 × 3.92 / 4.41 = 3.6×). The 6 E-cores add only another 0.9×: each has half the FMA units at a lower clock, and the P-cores slow further as the load rises.
- **The bound.** The whole program cannot run faster than its FMOPA phases alone, 2.25 ms. That caps the speedup at 2.9×, and the measured value is 2.51×. Efficiency is 0.25 at 10 threads because most of the machine's FP capacity sits in one shared unit.
- **Heterogeneous cores.** A block takes much longer on an E-core than on a P-core, so the loop uses `schedule(dynamic, 1)`. Compared with `schedule(static)` it is faster at every thread count I tested: 3.55 vs 4.37 ms at 4 threads, 2.72 vs 3.34 ms at 8, and 2.69 vs 3.05 ms at 10. macOS offers no thread pinning, so placement is the scheduler's choice.

Across inputs. Scaling is weakest at N = 1024 (1.72× at 10 threads). Packing K and the range check are serial and cost 0.04 ms there, already 13% of the 10-thread time, because they grow with N while the rest grows with N^2. At d = 128 the speedup is 1.85×, because the FMOPA phases take 68% of the single-thread time rather than 54%, and they do not scale. Memory bandwidth is not a limit in any case, as Q5 shows.

A NEON-only parallel version would scale better but be far slower: all 10 cores peak at 714 GFLOP/s, against the 1600+ GFLOP/s reached here, so scaling and speed pull in opposite directions on this chip.

Q5. Roofline

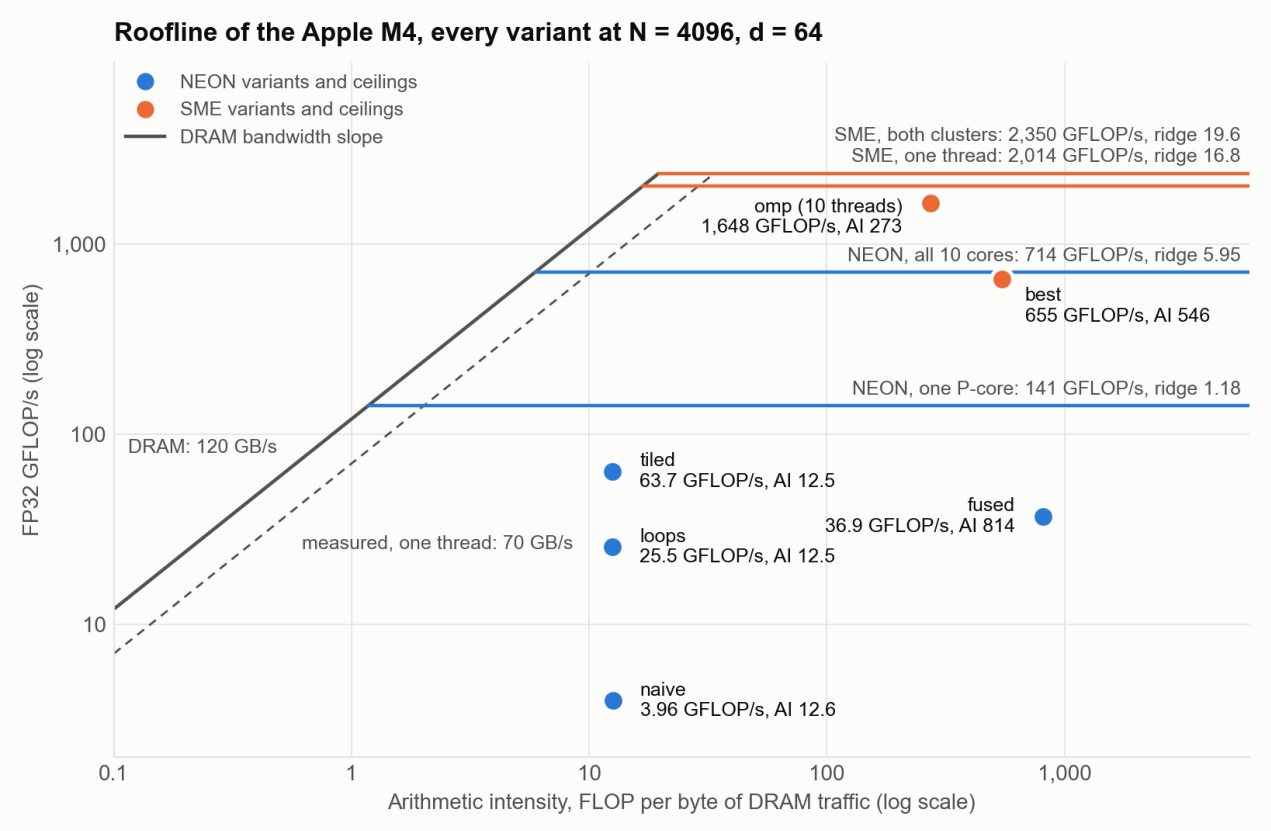


Figure 3. Attainable rate = min(peak, AI × bandwidth), from the Q1 numbers. Every variant at N = 4096, d = 64. The attained rates use the driver's FLOP count, so softmax lowers them. omp is shown at its fastest thread count.

How I computed arithmetic intensity

AI = FLOPs / bytes moved between DRAM and the CPU, for one `attention()` call starting with cold caches. FLOPs are the driver's $4N^2d$. macOS gives no access to memory-traffic counters without root, so I derived the bytes in two independent ways: an analytic count, and a trace-driven simulation of each variant's real access pattern through a model of the L1 and L2 (128 KiB 8-way, 16 MiB 16-way, 128-byte lines, LRU, write-allocate, write-back; SME accesses go straight to L2). The two agree, as Table 12 shows, and `results.csv` uses the simulated values.

Analytic count at N = 4096, where S (64 MB) cannot fit in L2:

- Q, K and V are each read once (4Nd bytes each) and O is written once, which with write-allocate means a read and a write-back. That gives 5 × 4Nd = 20Nd bytes for every variant. K, V, Q and O fit in L2, so repeated sweeps stay there.
- S adds 5 passes of $4N^2$ bytes in naive, loops and tiled: step 1 reads for ownership and writes back, softmax reads and writes back, and step 3 reads. So AI = $4N^2d$ / ($20N^2$ + 20Nd) ≈ d/5.
- loops adds K^T (8Nd). fused has only the 20Nd, so AI = N/5, independent of d.
- best adds the packed K (8Nd) and the 16 × N block buffer (128N). The omp model adds 128N for each extra thread, and 12Nd when the E-cluster's L2 must fetch K, V and Q as well.

Variant	Analytic bytes	Analytic (MB)	Simulated DRAM (MB)	AI (FLOP/B)	L2↔L1 traffic (GB)	AI at L2↔L1 (FLOP/B)
naive	20N² + 20Nd	340.8	340.8	12.6	144.21	0.03
loops	20N² + 28Nd	342.9	342.9	12.5	9.00	0.48
tiled	20N² + 20Nd	340.8	342.4	12.5	0.58	7.37
fused	20Nd	5.2	5.3	814	8.91	0.48
best	28Nd + 128N	7.9	7.9	546	0.14	31.27

Table 12. Traffic and intensity at N = 4096, d = 64. The analytic and simulated figures agree closely, which is good evidence for both. Simulated AI is used on the plot.

For the smaller inputs S fits in L2, so it costs only 2 passes rather than 5; N = 2048 is the boundary, where S is exactly L2-sized and only the simulation is reliable. The LRU model is pessimistic for naive's cyclic sweeps of V, so its 144 GB of L2↔L1 traffic is an upper bound.

What limits each variant

Variant	GFLOP/s	On the roofline	What limits it (evidence)
naive	3.96	Right of the ridge, at 2.8% of the NEON roof	Neither roofline resource. Dependent FP chains: 3.05-cycle FMADD per element in step 3, serial FADD in step 1 (Q2).
loops	25.5	Same AI as naive, far higher: reordering changes no DRAM traffic	L2 and memory-held accumulators: at 0.48 FLOP/B the L2 caps the products near 56 GFLOP/s, and they reach 29.
tiled	63.7	Same AI again, 45% of the roof	Compute. Products at 65% of peak; libm softmax is 30% of the time. DRAM has ample room: 12.5 FLOP/B × 120 GB/s = 1500 GFLOP/s.
fused	36.9	AI up 65×, yet lower than tiled	A limit the DRAM roofline cannot show: L2↔L1 bandwidth. 0.48 FLOP/B × 117 GB/s = 56 GFLOP/s; products reach 45.
best	655	4.6× above the NEON roof, 33% of the SME roof	SME instruction slots (products at 1,193 of a 1340 limit) and NEON softmax, 44% of the time. DRAM has no effect.
omp, 10 threads	1,648	70% of the two-cluster SME roof	One shared SME unit per cluster: FMOPA phases saturate near 1.25× (Q4).

Table 13. The limit on each variant.

Reading the plot. The NEON variants sit right of the NEON ridges (1.18 and 5.95 FLOP/B), and the SME variants right of the SME ridges (16.8 and 19.6), so DRAM bandwidth limits none of them. The chart does hold a lesson for SME. S-storing code has an AI of only 12.6, which is left of the SME ridges: 12.6 FLOP/B × 73 GB/s (measured single-thread) is about 920 GFLOP/s, under half the SME ceiling. So a version of `attn_best` that stored S would be pushed onto the memory slope: fusion matters little for NEON, but it is what makes the

SME ceiling reachable. That figure is a prediction from the model; I did not build that variant.

Appendix A. Microbenchmarks

The handout fixes the repository's contents, so the measurement tools are not in it. These are the kernels behind Table 4, written as inline assembly so that the compiler cannot change them. Each is timed over many iterations, taking the best of 9 runs, with the clock measured by the first listing.

```
/* Clock: 1000 dependent 1-cycle additions per iteration */
1:  .rept 1000
    add  x0, x0, x1
    .endr
    subs x2, x2, #1 ; b.ne 1b

/* FMA units and latency: k independent accumulators, here k = 4; vary k from 1 to 16 */
1:  .rept 12
    fmla v0.4s, v24.4s, v25.4s ; fmla v1.4s, v24.4s, v25.4s
    fmla v2.4s, v24.4s, v25.4s ; fmla v3.4s, v24.4s, v25.4s
    .endr
    subs x2, x2, #1 ; b.ne 1b

/* FMOPA: 4 tiles, one instruction = 512 FP32 FLOPs */
    smstart ; ptrue p0.s ; zero {za}
1:  .rept 16
    fmopa za0.s, p0/m, p0/m, z0.s, z1.s ; fmopa za1.s, p0/m, p0/m, z0.s, z1.s
    fmopa za2.s, p0/m, p0/m, z0.s, z1.s ; fmopa za3.s, p0/m, p0/m, z0.s, z1.s
    .endr
    subs x2, x2, #1 ; b.ne 1b
    smstop

/* Read bandwidth, NEON: 256 bytes per iteration, over each working-set size */
1:  ldp q0, q1, [x0]       ; ldp q2, q3, [x0, #32]   ; ldp q4, q5, [x0, #64]
    ldp q6, q7, [x0, #96]  ; ldp q16, q17, [x0, #128] ; ldp q18, q19, [x0, #160]
    ldp q20, q21, [x0, #192] ; ldp q22, q23, [x0, #224]
    add x0, x0, #256 ; subs x2, x2, #1 ; b.ne 1b
```

E-core results use `taskpolicy -b`, which confines a process to the E cluster at a low clock; instructions per cycle are the median over 201 adjacent clock and FMLA measurements, so a clock change cannot bias them. Thread counts across clusters use QoS classes, because macOS has no thread affinity.

Appendix B. Reproducing the results

```
brew install llvm            # the Makefile asks for the latest clang; Apple clang rejects -fopenmp
make all && make check       # build, time every variant on every input, compare with naive
OMP_NUM_THREADS=4 make omp   # one thread count
make best CFLAGS="-O3 -march=native -DNO_CLOCK_WARMUP"   # without the warm-up
```

`attn_tiled.c` and `attn_fused.c` need NEON, and `attn_best.c` and `attn_omp.c` need SME2 (Apple M4 or later), since the assignment asks for speed on my own machine rather than portability. Beyond the given inputs I tested all variants against naive on 27 other cases, with N and d not multiples of 4, 16 or 64, scores large enough to force the max-scan path, exp arguments far below −87, and V scaled by $10^{\pm 15}$. The worst error was 1.5×10^{-5}, within the 10^{-4} limit.